

AI 서비스 아키텍처 통합

Frontend, API, DB, LLM, RAG가 함께 동작하는 전체 흐름

오늘의 핵심 질문

AI 서비스는 LLM 하나로만 만들어지지 않습니다.

기획자는 다음 질문에 답할 수 있어야 합니다.

- 사용자는 어디에서 서비스를 이용하는가?
- 사용자의 요청은 어떤 경로로 이동하는가?
- 서비스 데이터는 어디에 저장되는가?
- AI는 어떤 정보를 보고 답변하는가?
- RAG는 언제 사용되고, LLM과 어떻게 연결되는가?

AI 서비스 아키텍처는 **사용자 요청이 데이터와 AI를 거쳐 결과로 돌아오는 흐름**입니다.

AI 서비스 아키텍처란?

아키텍처는 서비스 구성 요소들이 서로 어떻게 연결되는지 보여주는 구조입니다.

AI 서비스에서는 보통 다음 요소가 함께 움직입니다.

구성 요소	역할
Frontend	사용자가 보고 조작하는 화면
API	화면과 내부 기능을 연결하는 통로
DB	서비스 운영에 필요한 데이터를 저장
RAG	문서와 지식을 검색해 AI에게 전달
LLM	사용자의 질문에 자연어로 답변 생성

아키텍처를 이해하면 문제가 생겼을 때 어느 부분을 확인해야 하는지 알 수 있습니다.

전체 구조 한눈에 보기

AI 서비스의 기본 흐름은 다음과 같습니다.

사용자 화면 → API → DB / RAG → LLM → API → 사용자 화면

각 요소는 혼자 일하지 않습니다.

- Frontend는 사용자의 요청을 받음
- API는 요청을 해석하고 필요한 기능을 호출함
- DB는 사용자, 상품, 기록 같은 데이터를 제공함
- RAG는 문서 기반 근거를 찾아옴
- LLM은 데이터와 근거를 바탕으로 답변을 생성함

좋은 AI 서비스는 각 요소의 책임이 명확합니다.

Frontend의 역할

Frontend는 사용자가 직접 만나는 화면입니다.

AI 서비스에서 Frontend는 다음 역할을 합니다.

- 질문 입력창, 버튼, 결과 화면 제공
- 사용자 입력값을 API로 전달
- AI 답변, 출처, 경고 문구 표시
- 로딩 상태와 오류 상태 안내
- 사용자가 다음 행동을 할 수 있게 흐름 제공

Frontend는 단순한 화면이 아니라

AI 답변을 사용자가 이해하고 신뢰하게 만드는 접점입니다.

API의 역할

API는 Frontend와 내부 시스템을 연결하는 중간 통로입니다.

AI 서비스에서 API는 다음 일을 합니다.

- 사용자 요청을 받음
- 필요한 입력값이 있는지 확인함
- DB, RAG, LLM 중 필요한 기능을 호출함
- 결과를 정리해 Frontend로 돌려줌
- 오류가 발생했을 때 적절한 메시지를 전달함

API는 서비스 흐름을 조율하는 중심 역할을 합니다.

DB의 역할

DB는 서비스 운영 데이터를 저장하고 관리합니다.

AI 서비스에서 DB에는 다음 데이터가 들어갈 수 있습니다.

데이터 유형	예시
사용자 데이터	회원 정보, 설정, 권한
서비스 데이터	상품, 주문, 예약, 상담 기록
운영 데이터	로그, 평가 결과, 사용 이력
관리 데이터	카테고리, 상태값, 정책 설정

DB는 AI가 답변할 때 필요한 서비스 맥락을 제공할 수 있습니다.

LLM의 역할

LLM은 자연어를 이해하고 답변을 생성하는 AI 모델입니다.

AI 서비스에서 LLM은 다음 역할을 합니다.

- 사용자 질문의 의도를 이해함
- 전달받은 정보와 문서를 읽음
- 사용자가 이해하기 쉬운 답변을 생성함
- 요약, 분류, 추천, 설명 같은 언어 작업을 수행함

LLM은 답변을 만드는 역할을 하지만, 서비스의 모든 데이터를 직접 관리하지는 않습니다.

RAG의 역할

RAG는 LLM이 답변하기 전에 필요한 문서를 찾아주는 구조입니다.

RAG는 다음 상황에서 중요합니다.

- 답변에 근거 문서가 필요할 때
- 상품 정보, 정책, FAQ처럼 자주 바뀌는 정보가 있을 때
- LLM의 일반 지식만으로 답하기 어려울 때
- 사용자가 출처를 확인해야 할 때

RAG는 LLM에게 **답변에 사용할 근거 자료**를 전달하는 역할을 합니다.

DB와 RAG의 차이

DB와 RAG는 모두 데이터를 다루지만 목적이 다릅니다.

구분	DB	RAG
주요 목적	서비스 데이터를 정확히 저장하고 조회	질문과 관련된 문서 근거 검색
데이터 예시	회원, 주문, 상품 상태, 로그	FAQ, 설명서, 정책 문서, 상품 상세 문서
조회 방식	조건에 맞는 데이터 조회	의미가 가까운 문서 검색
AI와의 관계	서비스 맥락 제공	답변 근거 제공

LLM과 RAG의 관계

LLM과 RAG는 서로 다른 역할을 합니다.

구분	LLM	RAG
역할	답변 생성	근거 검색
강점	자연어 이해와 표현	문서 기반 정보 찾기
한계	근거 없이 말할 수 있음	검색된 문서 품질에 영향을 받음
함께 쓰는 이유	검색된 근거를 자연스럽게 설명하기 위해	LLM 답변의 신뢰도를 높이기 위해

RAG가 근거를 찾고, LLM이 그 근거를 사용해 답변합니다.

전체 서비스 흐름

사용자가 AI 기능을 사용할 때 흐름은 다음과 같습니다.

1. 사용자가 Frontend에서 질문을 입력한다
2. Frontend가 질문을 API로 보낸다
3. API가 사용자 정보와 요청 조건을 확인한다
4. API가 필요한 서비스 데이터를 DB에서 조회한다
5. API가 질문과 관련된 문서를 RAG로 검색한다
6. API가 DB 데이터와 RAG 문서를 LLM에 전달한다
7. LLM이 답변을 생성한다
8. API가 답변과 출처를 정리해 Frontend로 보낸다
9. Frontend가 사용자에게 결과를 보여준다

요청과 응답의 흐름

AI 서비스는 요청과 응답이 오가는 구조입니다.

단계	이동 방향	내용
요청	Frontend → API	사용자 질문과 필요한 정보 전달
조회	API → DB	서비스 데이터 확인
검색	API → RAG	질문과 관련된 문서 검색
생성	API → LLM	근거와 조건을 바탕으로 답변 요청
응답	API → Frontend	답변, 출처, 오류 메시지 전달

기획자는 이 흐름에서 어떤 정보가 언제 필요한지 정의해야 합니다.

예시 흐름

사용자가 다음과 같이 질문한다고 가정해 봅니다.

이 제품을 임산부가 먹어도 되나요?

서비스는 다음처럼 움직입니다.

- Frontend: 질문을 입력받고 API로 전달
- API: 상품명과 사용자 질문을 확인
- DB: 해당 상품의 기본 정보 조회
- RAG: 상품 상세 문서와 주의사항 문서 검색
- LLM: 검색된 근거를 바탕으로 답변 생성
- Frontend: 결론, 근거, 주의사항, 출처 표시

이렇게 역할을 나누면 답변이 어디서 만들어졌는지 추적할 수 있습니다.

각 요소의 책임 구분

아키텍처에서 중요한 것은 책임을 섞지 않는 것입니다.

요소	책임
Frontend	입력과 결과 표시
API	흐름 제어와 기능 연결
DB	정형 데이터 저장과 조회
RAG	문서 검색과 근거 제공
LLM	자연어 답변 생성

책임이 불명확하면 오류가 발생했을 때 원인을 찾기 어렵습니다.

오류가 생겼을 때 보는 위치

AI 서비스 오류는 여러 위치에서 생길 수 있습니다.

증상	확인할 위치
화면이 멈춤	Frontend, API 응답
사용자 정보가 틀림	DB 조회
근거가 이상함	RAG 검색 결과
답변이 과장됨	LLM 프롬프트
출처가 안 보임	API 응답 형식, Frontend 표시

아키텍처를 알면 "AI가 이상하다"가 아니라 어느 부분을 확인할지 나눌 수 있습니다.

기획자가 확인할 질문

AI 서비스 아키텍처를 검토할 때는 다음을 확인합니다.

- 사용자는 어떤 화면에서 어떤 행동을 하는가?
- API는 어떤 요청을 받고 무엇을 돌려주는가?
- DB에서 필요한 데이터는 무엇인가?
- RAG가 참고할 문서는 무엇인가?
- LLM에 전달할 정보와 답변 규칙은 무엇인가?
- 답변, 출처, 오류 메시지는 화면에 어떻게 표시되는가?
- 실패했을 때 사용자는 다음 행동을 알 수 있는가?

이 질문이 전체 서비스 흐름 설계의 기준이 됩니다.

오늘의 정리

AI 서비스 아키텍처는 여러 요소가 연결된 흐름입니다.

오늘의 핵심은 두 가지입니다.

1. Frontend, API, DB, LLM, RAG는 각각 다른 책임을 가진다
2. 전체 서비스 흐름은 사용자 요청이 데이터와 AI를 거쳐 다시 화면으로 돌아오는 과정이다

기획자의 역할은 기술을 모두 구현하는 것이 아니라

각 요소가 어떤 책임을 갖고 어떻게 연결되는지 설명할 수 있게 만드는 것입니다.